

Building standalone EpisodeSleuth binaries

EpisodeSleuth can be packaged into a **self-contained executable** that runs the Fluent desktop GUI **without Python installed** on the target machine. On Windows this produces an `.exe` ; on Linux/macOS it produces a native binary (no file extension). It uses [PyInstaller](#) and a single cross-platform build spec, `episodesleuth.spec` , driven by a small wrapper script per OS.

There are three ways to ship the app, from simplest to most polished:

Method	Best for	Script
Standalone binary (this doc)	Handing someone a runnable app to double-click; Linux/macOS distribution	<code>build_binary.ps1</code> (Windows) / <code>build_binary.sh</code> (Linux/macOS)
MSIX installer	Windows Store / signed enterprise deployment	<code>build_msix.ps1</code>
Source install	Developers, or users who already have Python	<code>install.ps1</code> / <code>pip install -e .</code>

Quick start

Windows

```
powershell -ExecutionPolicy Bypass -File .\build_binary.ps1
```

Output: `dist\EpisodeSleuth\EpisodeSleuth.exe` . Zip the whole `dist\EpisodeSleuth` folder to share it - the recipient unzips and double-clicks `EpisodeSleuth.exe` , no Python required.

Linux / macOS

```
./build_binary.sh
```

Output: `dist/EpisodeSleuth/EpisodeSleuth` (a native executable binary, no `.exe` extension). Distribute the whole `dist/EpisodeSleuth` folder.

Both scripts build inside a throwaway `.buildvenv` so your system Python stays clean, then run `PyInstaller` against `episodesleuth.spec` .

Build options

Both wrappers accept the same switches (via flags on Windows, environment variables on Linux/macOS):

Option	Windows	Linux/macOS	Effect
Bundle the Vosk model	<code>-BundleModel</code>	<code>BUNDLE_MODEL=1 ./build_binary.sh</code>	Packs <code>models\</code> into the bundle for a fully offline app (much larger). Off by default - the model is downloaded on first run / by <code>install.ps1</code> .
Single-file build	<code>-OneFile</code>	<code>ONEFILE=1 ./build_binary.sh</code>	Produces one executable file (<code>EpisodeSleuth.exe</code> on Windows, <code>EpisodeSleuth</code> on Linux/macOS) instead of a one-folder bundle. Simpler to share but slower to start (it unpacks to a temp dir each launch).

Example - fully offline single-file Windows build:

```
powershell -ExecutionPolicy Bypass -File .\build_binary.ps1 -BundleModel -OneFile
```

What ends up in the bundle

`episodesleuth.spec` deliberately keeps the bundle lean:

- **Included:** the `audio_fingerprint` package (engine + CLI + GUI), `PySide6` + `PySide6-Fluent-Widgets`, `Vosk`, `metaphone`, `Levenshtein`, `NumPy`, and `config.json` + the app icon (`packaging/app.ico`).
- **Excluded:** heavy scientific / ML / cloud packages that may exist in your build environment but are never imported by `EpisodeSleuth` (`torch`, `tensorflow`, `pyarrow`, `opencv`, `scipy`, `pandas`, `matplotlib`, `boto3`, `jupyter`, other Qt bindings, `tkinter`, ...). Without these excludes the bundle balloons to ~1.7 GB; with them a normal one-folder build is roughly **~400 MB** (dominated by `PySide6/Qt` and `Vosk`).

The entry point is `gui/__main__.py` (the same `python -m audio_fingerprint.gui` target), and the spec adds both the project root and its parent to `pathex` so

the mixed bare imports (`gui_config` , `engine` , `fingerprint_core`) and package-qualified imports (`audio_fingerprint.gui.*`) all resolve.

Prerequisites

- **Python 3.9+** on `PATH` (only on the build machine - not on the machines that run the finished binary).
- **FFmpeg** is still required at **run time** for decoding audio. It is an external tool and is intentionally not bundled; install it with `winget install Gyan.FFmpeg` (Windows) or your package manager (Linux/macOS), or run `install.ps1` first, which sets it up.

The wrapper scripts install PyInstaller and the project's `requirements.txt` into `.buildvenv` automatically.

Running / distributing

- **One-folder build (default):** distribute the entire `dist/EpisodeSleuth` folder (zip it). The executable and its `_internal` support files must stay together.
- **One-file build (`-OneFile`):** distribute the single executable file (`EpisodeSleuth.exe` on Windows, `EpisodeSleuth` on Linux/macOS).

On first run, if the Vosk speech model was not bundled, use the app's **Settings** page to download it (or run `install.ps1`).

Packaging for distribution

The `build_binary.*` scripts produce a raw `dist/EpisodeSleuth` folder. To hand someone a single downloadable file with a tidy installer, use the packaging wrappers - they build the binary (if needed) and wrap it up:

Platform	Script	Produces (in <code>dist/</code>)
Linux x64	<code>./package_linux.sh</code>	<code>EpisodeSleuth-<version>-linux-x64.tar.gz</code> and (if <code>dpkg-deb</code> is present) <code>episodesleuth_<version>_amd64.deb</code>
Windows x64	<code>powershell -ExecutionPolicy Bypass -File .\package_windows.ps1</code>	<code>EpisodeSleuth-<version>-windows-x64.zip</code>

Linux

```
./package_linux.sh          # reuse existing build, make tar.gz (+ .deb if possible)
./package_linux.sh --build  # force a fresh binary build first
./package_linux.sh --no-deb # tar.gz only
```

The `.tar.gz` contains the `EpisodeSleuth/` bundle plus `install.sh`, `uninstall.sh`, a `.desktop` launcher, an icon and a `README-INSTALL.txt`. The end user extracts it and runs `./install.sh` (current user) or `sudo ./install.sh` (system-wide into `/opt`, with an `episodesleuth` command on `PATH`). The optional `.deb` installs into `/opt/episodesleuth`, declares `ffmpeg` as a dependency, and registers an app-menu entry.

Windows

```
powershell -ExecutionPolicy Bypass -File .\package_windows.ps1
powershell -ExecutionPolicy Bypass -File .\package_windows.ps1 -Build -BundleModel
```

The `.zip` contains the `EpisodeSleuth\` bundle plus `Install-EpisodeSleuth.ps1` / `.bat` and a `README-INSTALL.txt`. The end user extracts it and either double-clicks `EpisodeSleuth.exe` directly, or runs `Install-EpisodeSleuth.bat` (normal = current user via LocalAppData; “Run as administrator” = all users via Program Files) to get Desktop + Start-menu shortcuts.

Full end-user instructions live in [INSTALL_LINUX.md](#) and [INSTALL_WINDOWS.md](#).

`package_windows.ps1` must be run on Windows (it needs PowerShell, `Compress-Archive`, and produces a Windows binary). Likewise `package_linux.sh` must be run on Linux. There is no cross-compilation - build each platform's package on that platform.

Code signing and SmartScreen

The standalone binaries and the `.zip` / `.tar.gz` are **not code-signed**. On Windows this means SmartScreen may show “Windows protected your PC” the first time the app runs; the user clicks **More info -> Run anyway**. That single click is the only friction for an unsigned standalone build - there is no certificate for the user to install.

If you want to remove even that warning, your options are:

Option	Approx. cost	Notes
Ship the unsigned <code>.zip</code>	Free	One “Run anyway” click. Simplest; recommended for small/personal projects.
Azure Trusted Signing	~\$10 / month	Cheapest legitimate signing. Requires a 3+ year-old verifiable org, or passing individual identity validation.
OV (Organization Validation) cert	~\$150-\$400 / year	Requires a hardware token / cloud HSM (mandatory since 2023). SmartScreen reputation builds over time.
EV (Extended Validation) cert	~\$300-\$700 / year	Instant SmartScreen reputation (no warning). Requires a registered legal organization.

Let’s Encrypt does not issue code-signing certificates - it only issues TLS/HTTPS certificates, which cannot sign applications. Do not go down that path.

MSIX and self-signed certificates

`build_msix.ps1` signs the `.msix` with a **self-signed** certificate. That is fine for testing, but every end user must first install (trust) the `.cer` into their Trusted People / Trusted Root store - an elevated (UAC) step that is inherent to the Windows trust model and cannot be removed for a self-signed package. You can streamline it by shipping the `.cer` alongside a one-line elevated command, e.g.:

```
Import-Certificate -FilePath .\EpisodeSleuth.cer -CertStoreLocation Cert:\LocalMachine\TrustedPeople
```

but you cannot eliminate it. For most people the **unsigned standalone `.zip` is actually less hassle than a self-signed MSIX**: unzip and run, one SmartScreen click, no certificate install and no admin step. Reach for a real signing certificate (or Azure Trusted Signing) only when you specifically need a polished, warning-free installer.

Relationship to the MSIX build

`build_msix.ps1` performs the same PyInstaller step and then wraps the result in a signed `.msix` using `packaging\AppxManifest.xml`. Use `build_binary.ps1` when you just want a runnable folder/executable, `package_linux.sh` / `package_windows.ps1` when you want a distributable archive with an installer, and `build_msix.ps1` when you need a proper Windows installer / Store package.

All share `episodesleuth.spec` conceptually, so the runtime behavior is identical.